Louis — Fase 1: Pre-Alfa Takeplan

Wat Louis moet bou

F2 · Admin Beheerpaneel API Eindpunte (28 Apr – 5 Mei) Die fondament van die hele webtoepassings. Louis moet 'n REST API bou wat die Admin beheerpaneel se basisfunksies hanteer — gebruikerslyste, profiel-ophaling, en basiese CRUD-operasies.

F1 · Gebruiker Aktief/Inaktief Logika (6 – 12 Mei) Die besigheidslogika agter `merkGebruikerOnaktief()` — wanneer 'n gebruiker onaktief gemerk word, moet die API ook outomaties die gekoppelde NFC-tag se `NFC_Is_Aktief` veld op `false` stel en 'n `Waarskuwings`-rekord skep.

F20 · Kampus Opsigter Rollogika (11 – 15 Mei) Die rolgebaseerde middleware wat seker maak 'n Kampus Opsigter kan slegs sy eie toegewese kampusse besigtig, en niks anders nie.

Voorvereistes — Wat moet klaar wees voordat Louis kan begin?

Louis se Taak	Wie	Wat moet klaar wees
F2 · Admin API	**Jaco**	Die `Gebruikers`, `Rolle`, en `Gebruikers_Rolle` tabelle moet in die databasis bestaan met die korrekte PKs en FKs — Louis kan nie `GET /gebruikers` skryf as die tabel nie bestaan nie. Jaco se eerste taak (NFC-skema) is nie 'n blokkeerder nie, maar die Gebruikers-tabelle (F1/F20/F21 in Jaco se ry) is krities.
F1 · Aktief/Inaktief	**Jaco**	Die `NFC_Tags.NFC_Is_Aktief` veld en die `Waarskuwings` tabel moet bestaan.
F1 · Aktief/Inaktief	**Steven**	Steven bou die NFC-tag registrasie API (F13) — Louis sal Steven se eindpunt moet kan aanroep om die NFC-tag te deaktiveer. Hulle moet 'n interne API-kontrak saamstel.
F20 · Opsigter Rollogika	**Jaco**	Die `Kampus_Opsigter` tabel en die koppeling tussen `Kampus_Opsigter` en `Kampus` (een-tot-baie verhouding) moet bestaan.
F20 · Opsigter Rollogika	**Connor**	Connor hoef niks klaar te hê vir Louis om die middleware te skryf nie — maar Connor sal Louis se voltooide rol-eindpunte nodig hê om die webnavigasie te toets.

Aanbevole Aanpak

Week 1 (28 Apr – 1 Mei): Kommunikeer eers, kode later

Louis moet op Dag 1 met Jaco sit en die databasisskema bevestig voordat hy 'n enkele reël kode skryf. Die grootste risiko in Fase 1 is dat Louis API-eindpunte bou gebaseer op 'n aanname van hoe die tabelle lyk, en dan alles moet herrfaktoreer toe Jaco klaar is. Spesifiek moet hulle die volgende saamstel:

- Watter velde is in `Gebruikers` (alle kolomname en tipes)
- Hoe `Gebruikers_Rolle` die koppeling tussen 'n gebruiker en sy rol hanteer
- Hoe die `Kampus_Opsigter` tabel lyk

Week 1 (28 Apr – 5 Mei): Bou die API-fondasie

Louis moet begin met 'n skoon API-raamwerk opset — roetes, middleware-struktuur, en fouthantering. Stel voor hy gebruik die volgende volgorde:

1. Stel die projek op (framework, databasisverbinding, omgewingsveranderlikes)
2. Skryf die `GET /gebruikers` eindpunt eerste — dit is die eenvoudigste en bevestig dat die databasisverbinding werk
3. Voeg dan `POST`, `PUT`, `DELETE` by vir gebruikerbestuur
4. Voeg basiese JWT-verifikasie by — selfs al is die rolle nog nie volledig nie, moet die token-stelsel van die begin af in plek wees sodat Connor se frontend dit kan gebruik

Week 2 (6 – 12 Mei): Aktief/Inaktief Logika

Hier is die belangrikste punt: Louis moet **nie** net 'n `is_aktief` veld opdateer nie. Die korrekte volgorde binne die API-oproep moet wees:

1. Merk `Gebruikers` rekord as inaktief
2. Soek die gekoppelde `NFC_Tags` rekord via `Gebruiker_ID`
3. Stel `NFC_Is_Aktief = false`
4. Skep 'n nuwe rekord in `Waarskuwings` met 'n beskrywing soos `"Gebruiker handmatig deur Admin onaktief gemerk"`
5. Stuur terug bevestiging

Al hierdie stappe moet in **een transaksie** gebeur — as stap 3 misluk, moet stap 1 en 2 ook teruggerol word. Dit is die tipe fout wat later moeilik is om op te spoor as dit nie van die begin af reg gedoen word nie.

Week 2–3 (11 – 15 Mei): Rol Middleware

Louis moet 'n herbruikbare middleware-funksie skryf wat op elke beskermde roete gebruik kan word. Die patroon moet só lyk:

```
versoek → onttrek JWT-token → bepaal rol →
  is dit Admin?          → laat deur
  is dit Kampus_Opsigter? → bevestig kampus_id pas by toegewese kampusse
  andersins              → 404 terug
```

Die kritieke detail vir die Kampus Opsigter is dat die middleware die `kampus_id` in die versoek moet vergelyk met die lys kampusse wat aan daardie opsigter toegewys is in die databasis — nie net die rol kontroleer nie.

Aanbevole Kommunikasie met die Span

- **Met Jaco**: Daaglikse kort insek — "is die `Gebruikers` tabel klaar?" Louis is die mees afhanklike van Jaco se vordering in hierdie fase.
- **Met Steven**: Stem vroeg saam op die interne API-kontrak vir NFC-deaktivering sodat beide se eindpunte bymekaar pas wanneer hulle integreer.
- **Met Connor**: Stuur vir Connor die API-dokumentasie (selfs net 'n eenvoudige lys van roetes en verwagte response-formate) so vroeg as moontlik — Connor kan dan die UI bou sonder om op Louis te wag.

Connor — Fase 1: Pre-Alfa Takeplan

Wat Connor moet bou

F2 · Admin Beheerpaneel UI (28 Apr – 5 Mei) Die visuele webkoppelvlak vir die Administrateur. Dit sluit in die hoofnavigasie, gebruikerslys-bladsy, en die basiese uitleg van die beheerpaneel. Dit is die eerste ding wat die kliënt gaan sien — dit moet professioneel lyk en die korrekte rolgebaseerde navigasiestruktuur volg soos in die sitemap uiteengesit.

F1 · Gebruiker Aktief/Inaktief UI (6 – 12 Mei) Die UI-komponente wat die Administrateur toelaat om 'n gebruiker as aktief of inaktief te merk — 'n bevestigingsdialoog, statusaanduiding per gebruiker, en visuele terugvoer wanneer die aksie uitgevoer word.

F20/F21 · Rolgebaseerde Navigasie (Web) (11 – 15 Mei) Die navigasiestelsel moet aanpas na gelang van die ingetekende gebruiker se rol. 'n Admin sien alles, 'n Kampus Opsigter sien slegs sy paneelborde. Dit moet op die sitemap gebaseer wees wat ons reeds opgestel het.

Voorvereistes — Wat moet klaar wees voordat Connor kan begin?

Connor se Taak	Wie	Wat moet klaar wees
F2 · Admin Beheerpaneel UI	**Louis**	Connor het Louis se API-roetes nodig om werklike data te vertoon. Maar — Connor moet **nie wag nie**. Hy moet met gesimuleerde data ("mock data") begin en later die werklike API-aanroepe inprop.
F2 · Admin Beheerpaneel UI	**Jaco**	Geen harde blokkeerder nie — Connor werk teen die API, nie die databasis direk nie. Maar Jaco se tabelstruktuur bepaal watter velde op die UI gewys word (bv. `GEB_Naam`, `GEB_Van`, `GEB_Epos`). Jaco moet die finale veldname vroeg bevestig.
F1 · Aktief/Inaktief UI	**Louis**	Louis se `PATCH /gebruikers/:id/status` eindpunt moet bestaan voordat Connor die UI-knoppie aan die werklike API kan koppel. Die UI kan egter vroeg gebou word met 'n mock-oproep.
F20/F21 · Rolgebaseerde Navigasie	**Louis**	Louis se JWT-token moet die gebruiker se rol insluit — Connor onttrek die rol uit die token om die korrekte navigasie te wys. Sonder dit kan Connor nie die rolgebaseerde vertakking toets nie.

F20/F21 · Rolgebaseerde Navigasie	**Steven**	Steven se rollogika vir Sekuriteit Wag (F21) moet bekend wees sodat Connor die mobiele navigasie ook korrek kan opstel.

Aanbevole Aanpak

Week 1 (28 Apr – 1 Mei): Projek Opset & Komponentbiblioteek

Connor moet die eerste dag gebruik om die frontend-projek behoorlik op te stel — dit spaar baie tyd later. Die sleutelkeuses wat vroeg gemaak moet word:

1. **Framework & roetebeheer** — bevestig die tegniese stapel met die span (React, Vue, ens.) en stel roetebeheer op vir beide die web- en mobiele toepassing
2. **Komponentstruktuur** — skep 'n basiese gidsstruktuur wat die drie stelsels weerspieël (Web/Admin, Mobiel/Student, Mobiel/Sekuriteit)
3. **Ontwerptelsel** — kies 'n konsekwente kleurpalet, tipografie en komponentbiblioteek sodat die UI deur alle fases eenvormig lyk
4. **Omgewingsveranderlikes** — stel 'n `API_BASE_URL` omgewingsveranderlike op van die begin af, sodat die skakel na Louis se API later met een verandering gedoen kan word

Week 1 (28 Apr – 5 Mei): Bou die Admin UI met Mock Data

Die sleutelinsig hier is dat Connor **parallel met Louis** moet werk deur mock data te gebruik. Hy moet 'n eenvoudige mock-laag skep:

```
// Tydelike mock — word later vervang met werklike API-oproep
const getGebruikers = async () => {
  if (USE_MOCK) return mockGebruikers;
  return await api.get('/gebruikers');
}
```

Die voordeel is tweeledig — Connor wag nie op Louis nie, en wanneer Louis se API gereed is, is dit net een reël wat verander. Die Admin UI moet die volgende bladsye insluit volgens die sitemap:

- Aanmeld-bladsy (met 'n tydelike hardgekodeerde token vir toetsing)
- Gebruikerslys met soek- en filterfunksie
- Gebruikerprofiel-bladsy met bewerkingsvermoë

Week 2 (6 – 12 Mei): Aktief/Inaktief UI

Die aktief/inaktief komponent is eenvoudig om te bou maar moet versigtig hanteer word vanuit 'n UX-oogpunt — 'n Administrateur moet nie per ongeluk 'n gebruiker kan deaktiveer nie. Connor moet die volgende patroon volg:

1. 'n Duidelike statusaanduiding op die gebruikerslys (groen/rooi simbool)
2. 'n "Merk Onaktief" knoppie wat 'n **bevestigingsdialoog** oopmaak met die gebruiker se naam en 'n waarskuwing
3. Na bevestiging — 'n laai-animasie terwyl die API-oproep plaasvind
4. Na sukses — die statusaanduiding opdateer **sonder** om die hele bladsy te herlaai
5. As die API-oproep misluk — 'n duidelike foutboodskap eerder as 'n stille mislukking

Week 2–3 (11 – 15 Mei): Rolgebaseerde Navigasie

Dit is Connor se mees tegniese taak in Fase 1. Die navigasiestelsel moet op die volgende patroon gebou word:

```
Na aanmelding:
  onttrek rol uit JWT-token
    → Admin          : wys volledige webnavigasie (sitemap 3.1 Web)
    → Kampus_Opsigter: wys slegs paneelbord-roetes
    → Personeel      : herlei na mobiele toepassing
    → Student        : herlei na mobiele toepassing
    → Sekuriteit_Wag : herlei na mobiele toepassing
```

Die kritieke detail is dat Connor **roetebewaking** moet implementeer — as 'n Kampus Opsigter probeer om handmatig na `/admin/gebruikers` te navigeer, moet die stelsel hom terugstuur na sy eie paneelbord. Dit moet nie net 'n UI-versteek wees nie, maar 'n werklike roete-blokkering.

Aanbevole Kommunikasie met die Span

- **Met Louis**: Vra op Dag 1 vir 'n lys van alle beplande API-roetes en hul verwagte request/response-formate — selfs net in 'n gedeelde dokument. Dit laat Connor mock data skep wat die werklike struktuur weerspieël en voorkom dat hy later alles moet herrfaktoreer.
- **Met Jaco**: Bevestig die finale veldname in die `Gebruikers` tabel vroeg — veral `GEB_Profiel_Foto` aangesien dit 'n spesiale hanteringstrategie vereis (URL vs. base64).
- **Met Steven**: Stem saam op die navigasie-struktuur vir die mobiele toepassing se rolle (Student, Personeel, Sekuriteit Wag) sodat Connor die roetebewaking vir alle 5 rolle gelyktydig kan implementeer.

Steven — Fase 1: Pre-Alfa Takeplan

Wat Steven moet bou

F2 · Gebruiker CRUD API Eindpunte (28 Apr – 5 Mei) Waar Louis die Admin beheerpaneel se hoëvlak eindpunte bou, fokus Steven op die kernvlak gebruiker-databasis operasies — die `POST /gebruikers`, `GET /gebruikers/:id`, `PUT /gebruikers/:id`, en `DELETE /gebruikers/:id` eindpunte. Dit is die fondament waarop Louis se beheerpaneel-logika en Connor se UI albei staatmaak.

F13 · NFC-tag Registrasie API (6 – 12 Mei) Die eindpunte wat 'n fisiese NFC-tag aan 'n gebruikersprofiel koppel. Dit sluit in die skep van 'n nuwe `NFC_Tags` rekord, die koppeling van `Gebruiker_ID` aan die tag, en die validering dat 'n gebruiker nie meer as een aktiewe NFC-tag op 'n slag het nie.

F21 · Sekuriteit Wag Rollogika (11 – 15 Mei) Die API-middleware en eindpunte spesifiek vir die Sekuriteit Wag rol — watter roetes is toeganklik, watter data kan hulle sien, en hoe hul sessie verskil van 'n Admin of Kampus Opsigter se sessie.

Voorvereistes — Wat moet klaar wees voordat Steven kan begin?

Steven se Taak	Wie	Wat moet klaar wees
F2 · Gebruiker CRUD	**Jaco**	Die `Gebruikers` tabel moet bestaan met alle finale kolomme — `GEB_Naam`, `GEB_Van`, `GEB_ID_Nommer`, `GEB_Epos`, `GEB_Wagwoord`, `GEB_Selfoon_Nr`, `GEB_Profiel_Foto`, `Rol_ID`. Steven kan nie 'n `POST /gebruikers` skryf sonder om te weet watter velde verpligtend is en watter uniek moet wees (bv. `GEB_Epos` en `GEB_ID_Nommer`).
F2 · Gebruiker CRUD	**Louis**	Louis bou die JWT-verifikasie stelsel — Steven moet weet hoe die token gestruktureer is sodat sy eindpunte dieselfde verifikasiepatroon gebruik. Hulle moet op Dag 1 'n gedeelde auth-middleware kontrak saamstel.
F13 · NFC-tag Registrasie	**Jaco**	Die `NFC_Tags` tabel moet bestaan met `NFC_Tags_ID`, `NFC_UID`, `Gebruiker_ID`, `NFC_Tipe`, en `NFC_Is_Aktief`. Spesifiek moet Jaco bevestig of `NFC_UID` 'n unieke indeks het — dit bepaal hoe Steven duplikaat-registrasie hanteer.

F13 · NFC-tag Registrasie	**Louis**	Louis se aktief/inaktief logika (F1) roep Steven se NFC-eindpunt aan om 'n tag te deaktiveer — hulle moet vroeg saamstem op die presiese eindpunt-naam, parameters, en response-formaat sodat die integrasie glad verloop.
F21 · Sekuriteit Wag Rollogika	**Jaco**	Die `Sekuriteit_Wag` tabel met `SEK_Nommer` moet bestaan, en die koppeling met `Gebruikers` via oorerwing moet in die databasis weerspieël word.
F21 · Sekuriteit Wag Rollogika	**Louis**	Louis se rol-middleware raamwerk moet in plek wees — Steven brei dit uit vir die Sekuriteit Wag rol eerder as om 'n aparte stelsel te bou. Duplikasie van auth-logika is 'n groot risiko as hulle nie vroeg koordineer nie.

Aanbevole Aanpak

Week 1 (28 Apr – 1 Mei): Koördineer met Louis voor enige kode

Steven en Louis se werk oorvleuel meer as enige ander paar in die span. Beide bou API eindpunte, beide gebruik dieselfde auth-middleware, en Louis se F1-logika is direk afhanklik van Steven se F13-eindpunt. Die gevaar is dat hulle onafhanklik verskillende patrone gebruik en dan 'n integrasieprobleem in Week 2 ontdek.

Op Dag 1 moet Steven en Louis saam besluit op:

1. **Projekstruktuur** — een gedeelde backend-repo of twee afsonderlike? Watter gidstruktuur?
2. **Auth-middleware** — een gedeelde `verifyToken` en `checkRol` funksie wat albei gebruik
3. **Fouthantering patroon** — dieselfde response-formaat vir foute regdeur die API
4. **Naamkonvensies** — roete-name, veranderlike name, databasis-kolomverwysings

```
// Voorbeeld van gedeelde patroon wat albei moet gebruik
// eerder as elkeen sy eie implementasie
{
  "sukses": true,
  "data": { ... },
  "fout": null
}
```

Week 1 (28 Apr – 5 Mei): Bou die Gebruiker CRUD Fondament

Steven moet die CRUD eindpunte in 'n spesifieke volgorde bou om die span so vinnig moontlik te ontsluit:

1. **`GET /gebruikers` en `GET /gebruikers/:id` eerste** — Connor en Louis het hierdie nodig om hul eie werk te kan toets. Selfs 'n basiese implementasie met hardgekodeerde data is beter as niks.
2. **`POST /gebruikers`** — voeg robuuste validering by van die begin af:
    - `GEB_Epos` moet uniek wees — kontroleer voor invoeging
    - `GEB_ID_Nommer` moet uniek wees — kontroleer voor invoeging
    - `GEB_Wagwoord` moet gehash word — **nooit as gewone teks gestoor nie**
3. **`PUT /gebruikers/:id`** — gedeeltelike opdatering moet ondersteun word (nie alle velde verpligtend nie)
4. **`DELETE /gebruikers/:id`** — hier moet Steven met Louis koördineer, want 'n gebruiker word nooit werklik uitgevee nie — dit word eerder onaktief gemerk (F1). Die `DELETE` eindpunt moet dus eerder 'n sagte verwydering implementeer.

Die wagwoordhashing is 'n detail wat baie spanne vergeet in die pre-alfa fase en dan later moet herrfaktoreer. Steven moet dit reg van die begin af insit:

```
// Voor stoor
const gehashedWagwoord = await bcrypt.hash(wagwoord, 12);

// Nooit die wagwoord terugstuur in 'n response nie
const { GEB_Wagwoord, ...veiligeProfiel } = gebruiker;
return veiligeProfiel;
```

Week 2 (6 – 12 Mei): NFC-tag Registrasie

Die NFC-registrasie API is Steven se mees kritiese bydrae in Fase 1 omdat Louis se F1-logika direk hiervan afhang. Steven moet die volgende scenario's hanteer:

Gelukkige pad — nuwe tag word geregistreer:

1. Ontvang `NFC_UID` en `Gebruiker_ID`
2. Kontroleer of `NFC_UID` reeds bestaan — as ja, fout terug
3. Kontroleer of gebruiker reeds 'n aktiewe tag het — as ja, moet die ou tag eers onaktief gemerk word
4. Skep nuwe `NFC_Tags` rekord met `NFC_Is_Aktief = true`
5. Stuur bevestiging terug

Deaktivering pad — word deur Louis se F1-logika aangeroep:

```
// Eindpunt wat Louis gebruik
PATCH /nfc-tags/:gebruikerID/deaktiveer

// Steven se implementasie moet:
// 1. Die aktiewe tag vir daardie gebruiker vind
// 2. NFC_Is_Aktief = false stel
// 3. Bevestiging terug stuur
// 4. GEEN Waarskuwings rekord hier skep nie —
//    dit is Louis se verantwoordelikheid
```

Die laaste punt is belangrik — die verantwoordelikheidsverdeling moet duidelik wees. Steven deaktiveer die tag, Louis skep die waarskuwing. As albei dit probeer doen, kry jy duplikaat waarskuwings.

Week 2–3 (11 – 15 Mei): Sekuriteit Wag Rollogika

Steven moet die Sekuriteit Wag rol as 'n uitbreiding van Louis se bestaande middleware bou, nie as 'n aparte stelsel nie. Die Sekuriteit Wag het 'n unieke toegangsprofiel:

- **Mag**: Skandeer gebruikers, besigtig basiese profieldata, skep toegangsloglêers, hanteer besoekers
- **Mag nie**: Gebruikers wysig, NFC-tags uitreik, paneelborde besigtig, enige admin-funksies

Die middleware moet soos volg werk:

```
Versoek ontvang
  → Verifieer JWT (gedeelde Louis/Steven funksie)
  → Is rol = Sekuriteit_Wag?
    → JA: laat slegs toe:
        GET /gebruikers/:id (beperkte velde — geen wagwoord, geen finansiële data)
        POST /toegangs-logs
        POST /besoekers
        GET /besoekers
    → NEE: stuur 403 terug
```

Die beperkte velde vir `GET /gebruikers/:id` is belangrik — die Sekuriteit Wag moet genoeg sien om 'n persoon te identifiseer (naam, foto, ID-nommer, NFC-status) maar nie sensitiewe velde soos `GEB_Wagwoord` of `GEB_Epos` nie.

Aanbevole Kommunikasie met die Span

- **Met Louis**: Sit daagliks vir 10 minute saam — hierdie twee se werk is die nouverweefste in die hele span. Enige verskil in hul patrone skep integrasieprobleme later.
- **Met Jaco**: Steven moet Jaco se vordering op die `NFC_Tags` tabel aktief dophou — dit is sy Week 2 se blokkeerder. As Jaco agter raak, moet Steven proaktief aanmeld en nie net wag nie.
- **Met Connor**: Stuur vir Connor 'n eenvoudige API-verwysing sodra `GET /gebruikers` en `GET /gebruikers/:id` werk — al is dit net 'n gedeelde Notion-dokument of README. Connor kan dan ophou met mock data en met die werklike API begin toets.

Jaco — Fase 1: Pre-Alfa Takeplan

Wat Jaco moet bou

F3 · NFC-tag Databasisskema Ontwerp (28 Apr – 2 Mei) Die volledige `NFC_Tags` tabel met alle kolomme, tipes, beperkings, en indekse. Dit sluit in die unieke indeks op `NFC_UID`, die vreemde sleutel na `Gebruikers`, en die koppeling na `Voertuig`. Hierdie tabel word in Fase 2 intensief gebruik — as die skema nou verkeerd ontwerp word, betaal die span later 'n hoë prys in migrasies.

F13 · NFC-Uitreiking Tabelstruktuur (5 – 8 Mei) Die ondersteuning vir die registrasieproses — seker maak dat die `NFC_Tags` tabel die volledige lewensiklus van 'n tag kan hanteer, van uitreiking tot deaktivering, met die nodige ouditspoortjie om te weet wanneer 'n tag verander het.

F1 · Gebruikers-Lewensiklus Tabelle (11 – 13 Mei) Die `Gebruikers` tabel met alle velde, beperkings, en die korrekte hantering van die aktief/inaktief status. Dit sluit ook die `Waarskuwings` tabel in aangesien dit direk aan die onaktief-merk proses gekoppel is.

F20/F21 · Rolle & Gebruikers Tabelle (14 – 16 Mei) Die `Rolle`, `Gebruikers_Rolle`, `Kampus_Opsigter`, en `Sekuriteit_Wag` tabelle met alle verhoudings en beperkings. Dit is die laaste stuk van die Fase 1 databasis-fondasie.

Voorvereistes — Wat moet klaar wees voordat Jaco kan begin?

Jaco is uniek in die span — hy het die **minste eksterne voorvereistes** maar is terselfdertyd die **grootste blokkeerder** vir almal anders. Sy werk hang hoofsaaklik af van besluite, nie van ander mense se kode nie.

Jaco se Taak	Wie	Wat moet klaar wees
F3 · NFC-tag Skema	**Jaco self**	Die ERD moet finaal wees voor enige tabel geskep word. Die ERD wat ons reeds opgestel het is die bloudruk — Jaco moet dit een keer deeglik deurlees en enige laaste vrae op Dag 1 uitklaar voor hy begin kodeer.
F3 · NFC-tag Skema	**Span besluit**	Die databasis-tegnologie moet op Dag 1 bevestig word — PostgreSQL, MySQL, ens. Dit beïnvloed hoe Jaco tipes, beperkings en indekse skryf.

F1 · Gebruikers Tabelle	**Louis & Steven**	Jaco moet weet watter velde die backend gaan benodig op `Gebruikers` — veral of daar enige bykomende velde is wat Louis of Steven beplan wat nie in die ERD is nie. 'n Vinnige gesels op Dag 1 voorkom 'n migrasie later.
F20/F21 · Rolle Tabelle	**Connor**	Connor se rolgebaseerde navigasie hang af van hoe rolle in die databasis gestoor word. Jaco moet bevestig of rolle as 'n vaste lys in die `Rolle` tabel gestoor word, sodat Connor weet of hy teen 'n ID of 'n naam moet kontroleer.

Aanbevole Aanpak

Voor Dag 1: Die ERD is Jaco se Kontrak met die Span

As projekleier is Jaco se databasis-ontwerp nie net 'n tegniese taak nie — dit is die kontrak waarteen die hele span ontwikkel. Elke tabel wat Jaco skep definieer wat Louis en Steven kan bou en watter data Connor kan vertoon. Voordat Jaco die eerste `CREATE TABLE` skryf, moet hy die finale ERD een keer volledig deurlees en die volgende vrae beantwoord:

- Is alle verhoudings korrek — veral die een-tot-baie tussen `Kampus_Opsigter` en `Kampus`?
- Is alle vreemde sleutels in die regte rigting?
- Watter velde moet uniek wees (`GEB_Epos`, `GEB_ID_Nommer`, `NFC_UID`, `VOE_Nommerplaat`)?
- Watter velde mag nooit `NULL` wees nie?

Week 1 (28 Apr – 2 Mei): Bou die NFC Skema Eerste

Dit mag teenintuïtief lyk — waarom NFC voor Gebruikers? Die rede is dat `NFC_Tags` afhang van `Gebruikers`, maar `Gebruikers` het geen afhanklikheid van `NFC_Tags` nie. Deur NFC eerste volledig te ontwerp, dwing Jaco homself om die hele koppeling te deurdenk voor hy `Gebruikers` bou. Dit voorkom die situasie waar hy `Gebruikers` bou, dan `NFC_Tags` byvoeg, en besef die struktuur moet verander.

Die `NFC_Tags` tabel moet die volgende beperkings hê van die begin af:

```
CREATE TABLE NFC_Tags (
    NFC_Tags_ID     SERIAL PRIMARY KEY,
    NFC_UID         VARCHAR(64)  NOT NULL UNIQUE,  -- fisiese tag se ID
    Gebruiker_ID    INT          REFERENCES Gebruikers(Gebruiker_ID),
    NFC_Tipe        VARCHAR(32)  NOT NULL,         -- 'STUDENT', 'VOERTUIG', 'LOKASIE'
    NFC_Is_Aktief   BOOLEAN      NOT NULL DEFAULT TRUE,
    -- Ouditspoor
    Geskep_Op       TIMESTAMP    NOT NULL DEFAULT NOW(),
    Gewysig_Op      TIMESTAMP    NOT NULL DEFAULT NOW()
);

-- Indeks vir vinnige opsoek tydens skandering
CREATE INDEX idx_nfc_uid ON NFC_Tags(NFC_UID);
CREATE INDEX idx_nfc_gebruiker ON NFC_Tags(Gebruiker_ID);
```

Die ouditspoor (`Geskep_Op`, `Gewysig_Op`) is iets wat baie spanne in pre-alfa vergeet en later betreur — veral vir 'n sekuriteitstelstel waar jy moet kan bewys wanneer 'n tag geaktiveer of gedeaktiveer is.

Week 1 (5 – 8 Mei): NFC-Uitreiking Tabelstruktuur

Jaco moet seker maak die `NFC_Tags` tabel ondersteun die volledige uitreikingsproses. Die kritieke besigheidsreël hier is dat een gebruiker slegs **een aktiewe tag** op 'n slag mag hê, maar historiese tags moet behoue bly vir ouditdoeleindes:

```
-- Voorkom twee aktiewe tags vir dieselfde gebruiker
-- via 'n gedeeltelike unieke indeks
CREATE UNIQUE INDEX idx_een_aktiewe_tag_per_gebruiker
    ON NFC_Tags(Gebruiker_ID)
    WHERE NFC_Is_Aktief = TRUE;
```

Hierdie enkele reël kode voorkom 'n hele klas van foute wat andersins net tydens toetsing ontdek sou word. Dit is die tipe databasis-beperking wat goedkoper is om nou in te sit as om later in die toepassing se logika te hanteer.

Week 2 (11 – 13 Mei): Gebruikers-Lewensiklus Tabelle

Die `Gebruikers` tabel is die sentrale tabel van die hele stelsel — byna elke ander tabel het 'n vreemde sleutel daarna. Jaco moet dit met uiterste sorg bou:

```
CREATE TABLE Gebruikers (
    Gebruiker_ID    SERIAL PRIMARY KEY,
    GEB_Naam        VARCHAR(100) NOT NULL,
    GEB_Van         VARCHAR(100) NOT NULL,
    GEB_ID_Nommer   VARCHAR(13)  NOT NULL UNIQUE,  -- SA ID: 13 syfers
    GEB_Epos        VARCHAR(255) NOT NULL UNIQUE,
    GEB_Wagwoord    VARCHAR(255) NOT NULL,          -- gehash deur Steven
    GEB_Selfoon_Nr  VARCHAR(15),
    GEB_Profiel_Foto VARCHAR(500),                  -- URL na lêer
    Rol_ID          INT          NOT NULL REFERENCES Rolle(Rol_ID),
    Is_Aktief       BOOLEAN      NOT NULL DEFAULT TRUE,
    Geskep_Op       TIMESTAMP    NOT NULL DEFAULT NOW(),
    Gewysig_Op      TIMESTAMP    NOT NULL DEFAULT NOW()
);
```

Die `Waarskuwings` tabel moet ook hier gebou word aangesien Louis dit onmiddellik nodig het vir sy aktief/inaktief logika:

```
CREATE TABLE Waarskuwings (
    Waarskuwing_ID  SERIAL PRIMARY KEY,
    Gebruiker_ID    INT         NOT NULL REFERENCES Gebruikers(Gebruiker_ID),
    Lokasie_ID      INT         REFERENCES Lokasies(Lokasie_ID),
    WAA_Beskrywing  TEXT        NOT NULL,
    WAA_Status      VARCHAR(32)  NOT NULL DEFAULT 'OOP',  -- 'OOP', 'ERKEN', 'GESLUIT'
    WAA_Tydstempel  TIMESTAMP    NOT NULL DEFAULT NOW()
);
```

Let op die `WAA_Status` veld met 'n `DEFAULT` waarde — dit beteken Steven en Louis hoef nie die status elke keer te spesifiseer wanneer hulle 'n waarskuwing skep nie, maar dit kan later deur die Admin bygewerk word.

Week 2–3 (14 – 16 Mei): Rolle & Gebruikers Tabelle

Die rolle-stelsel moet as 'n vaste datastel ingevul word tydens die migrasie — nie as dinamiese data nie. Die vyf rolle moet van die begin af in die databasis bestaan:

```
CREATE TABLE Rolle (
    Rol_ID    SERIAL PRIMARY KEY,
    ROL_Naam  VARCHAR(50) NOT NULL UNIQUE
);

-- Saai data — loop saam met die skema-migrasie
INSERT INTO Rolle (ROL_Naam) VALUES
    ('Administrateur'),
    ('Kampus_Opsigter'),
    ('Personeel'),
    ('Student'),
    ('Sekuriteit_Wag');
```

Die subklastabelle vir `Kampus_Opsigter` en `Sekuriteit_Wag` moet die oorerwingspatroon van die ERD weerspieël:

```
CREATE TABLE Kampus_Opsigter (
    Gebruiker_ID    INT PRIMARY KEY REFERENCES Gebruikers(Gebruiker_ID),
    KAMO_Nommer     VARCHAR(20) NOT NULL UNIQUE
);

CREATE TABLE Sekuriteit_Wag (
    Gebruiker_ID    INT PRIMARY KEY REFERENCES Gebruikers(Gebruiker_ID),
    SEK_Nommer      VARCHAR(20) NOT NULL UNIQUE
);
```

Die `Gebruiker_ID` as primêre sleutel wat terselfdertyd 'n vreemde sleutel is, is die klassieke SQL-oorerwingspatroon — dit dwing 'n een-tot-een verhouding af sonder duplikasie.

Jaco se Verantwoordelikheid as Projekleier

Buite sy databasis-werk het Jaco 'n tweede rol wat net so belangrik is. Hy moet:

'n Gedeelde omgewing opset — databasverbindingstrings, omgewingsveranderlikes, en 'n `docker-compose.yml` of soortgelyke opstelling sodat Louis, Steven, en Connor almal teen dieselfde ontwikkelingsdatabasis kan werk sonder konflikte.

Migrasielêers bestuur — elke skema-verandering moet as 'n genommerde migrasielêer gestoor word eerder as om direk teen die databasis uitgevoer te word:

```
migrations/
  001_nfc_tags.sql
  002_gebruikers.sql
  003_waarskuwings.sql
  004_rolle.sql
```

Dit beteken as Louis of Steven 'n fout kry, kan hulle die databasis presies herskep sonder om vir Jaco te wag.

'n Spanwye databasis-verwysing onderhou — 'n lewendige dokument (README of wiki-bladsy) wat altyd die huidige tabelstruktuur, veldname, en tipes weerspieël. Dit is die enkele dokument wat die span raadpleeg as hulle onseker is oor 'n veldnaam of tipe — dit elimineer die situasie waar Louis `GEB_Naam` gebruik en Steven `GEB_naam` (hooflettersensitief) en alles breek.

Aanbevole Kommunikasie met die Span

- **Met Louis**: Gee hom 'n vroeë sein sodra `Gebruikers` en `Waarskuwings` gereed is — dit is sy Week 2 se blokkeerder en elke dag wat hy wag is 'n dag wat hy nie kan toets nie.
- **Met Steven**: Bevestig die `NFC_UID` unieke indeks en die gedeeltelike unieke indeks vir aktiewe tags vroeg — Steven se registrasie-logika hang af van hierdie databasis-beperkings.
- **Met Connor**: Stuur die finale veldname vir `Gebruikers` sodra dit vasgestel is — veral `GEB_Profiel_Foto` se formaat (URL of base64) sodat Connor die UI korrek kan ontwerp.
- **As Projekleier**: Skeduleer 'n kort spanvergadering elke Maandag-oggend om blokkeerders vroeg te identifiseer — nie om vordering te rapporteer nie, maar om te vra "wag jy op iemand anders?" en dit onmiddellik op te los.